

Accelerating Distributed Digital Ising Machines

Tong Wu*, Ziyad Alswaidan*, Shreya Mukherjee*, Ting-Yu Lee*, Daniel Xu*, Atharva Raut*,
Harry Tye†, Abhinav K. Puthran*, Kemal Selcuk‡, Navid A. Aadit‡,
Ken Mai*, Prashanth Mohan†, Kerem Y. Camsari‡, Tathagata Srimani*

*Carnegie Mellon University †University of Pittsburgh ‡University of California, Santa Barbara

Abstract—Ising machines are dedicated accelerators for combinatorial optimization and probabilistic sampling using Ising-like spin dynamics. The problems that motivate them require millions of spins, beyond the on-chip capacity of one device, so practical systems must distribute the computation. Prior work reports throughput at isolated operating points, leaving the performance bottleneck and platform crossovers to be determined by experiment. We present the first roofline analysis of Ising workloads and a calibrated cost model that decomposes digital Ising sweep time into compute, memory, synchronization, and inter-device communication. The model has 4.8% mean error across three GPU generations, an FPGA, and an ASIC at problem sizes up to one million spins. The roofline analysis finds low arithmetic intensity, and measurements show a sharp performance loss when the working set spills beyond on-chip storage. Roofline-guided data placement yields up to $4.2\times$ speedup. We then propose two system-level acceleration approaches. First, bounded-staleness sampling reduces synchronization frequency through multi-rate bounded staleness within a partition and cross-partition bounded staleness across a partition boundary, reducing time-to-solution (TTS) by up to $3.16\times$ on three benchmarks. Second, phase-offset pipelining overlaps boundary exchange with interior computation whenever a link satisfies a latency-transparency criterion, yielding up to $1.25\times/1.74\times/2.01\times$ sweep speedup on an H200 pair, a ZCU106 pair, and a four-board XEM8320 2×2 mesh. We also implement the streaming dataflow on a fabricated four-chip 28 nm ASIC that sustains 28.1 Gflips/s at 4.81 pJ/flip.

I. INTRODUCTION

Combinatorial optimization problems such as logistics routing, resource scheduling, and probabilistic machine-learning inference are ubiquitous, and their underlying NP-hard structure makes them costly to solve at scale [1]. Probabilistic hardware already reaches several of these classes, training deep Boltzmann networks [2], running diffusion-based generative models [3], and competing with leading quantum annealers on hard optimization instances [4]. The Ising model has become a widely used abstraction for such problems: binary variables are represented as *spins*, and pairwise couplings are chosen so that low-energy spin configurations correspond to high-quality solutions [1], [5]–[7]. Hardware architectures that exploit this abstraction are broadly called Ising machines.

Ising machines can be realized in many ways. Some implementations embed the dynamics directly in physical substrates, including superconducting quantum annealers with thousands of qubits [8]–[10] and optical coherent Ising machines with up to 100,000 spins [11]. Others implement the spin dynamics

digitally, including probabilistic-bit machines in which each spin is realized as a stochastic binary unit whose probability is set by its inputs [12]. This paper focuses on digital Ising machines [13], [14] implemented on GPUs, FPGAs, and ASICs, where the computational kernel is explicit, measurable, and comparable across platforms.

Mapping a problem onto such hardware follows a common workflow: the problem is expressed as an Ising model, its irregular interaction graph is transformed into a sparse, regular, hardware-friendly graph, and the resulting spin-update workload is executed on a target platform (Fig. 1). Realizing the resulting acceleration at practical problem scales requires millions of spins, which exceed the on-chip capacity of any single digital accelerator [15]. Thus, irrespective of the hardware implementation, scaling ultimately forces the Ising machine to become distributed. The central question is therefore not simply which device (GPU, FPGA, or ASIC) is faster, but which resource limits scaling on each implementation. One might expect arithmetic to dominate, since each sweep performs local-field accumulation and stochastic spin updates. In practice, it does not. Performance is governed by where couplings reside in the memory hierarchy, whether the working set fits in fast on-chip storage, how often spin updates must synchronize, and how much boundary state must cross device boundaries. These costs depend strongly on graph topology, so the same Ising sweep can be memory-bound on a GPU, synchronization-sensitive under global update barriers, and communication-bound once partitioned across devices.

Our contributions separate the workload analysis from the two system-level acceleration approaches:

- **Modeling the distributed Ising sweep** with a calibrated four-term decomposition into compute, memory, synchronization, and inter-device communication, which reproduces measured runtime to 2.6–5.2% mean error across Ada, Blackwell, and Hopper GPUs up to million-spin scale, and predicts statically scheduled FPGA and ASIC throughput from synthesis-time constants.
- **The first roofline analysis of Ising workloads**, showing that arithmetic intensity remains low as graph degree grows and that performance degrades sharply when the working set spills beyond on-chip storage. Storage format, residency tier, and datapath precision determine the crossover; the measured data-placement choices yield $1.2\text{--}4.2\times$ speedup and a geometric-mean $3.4\times$ TTS reduction at an identical energy target.

Corresponding author: tsrimani@andrew.cmu.edu.

This work has been submitted to the IEEE for possible publication.

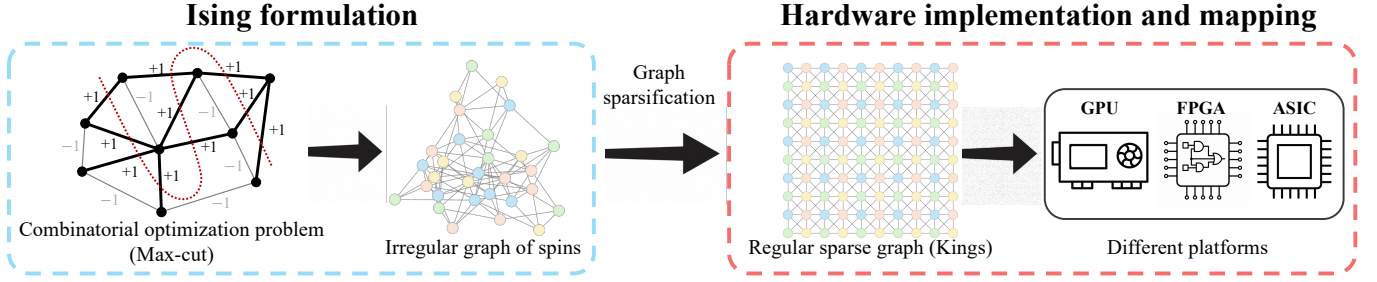


Fig. 1. Workflow for mapping a combinatorial optimization problem onto a digital Ising machine. This work focuses on hardware implementation and mapping (red box) across GPU, FPGA, and ASIC platforms.

- **Two forms of bounded-staleness sampling**: multi-rate bounded staleness, in which interior p-bits observe stale boundary neighbors within a partition, and cross-partition bounded staleness, in which boundary p-bits observe stale neighbors across a partition cut. Reducing synchronization frequency trades solution quality for sweep rate and gives $1.75\text{--}3.16\times$ and $1.84\text{--}2.17\times$ TTS reductions, respectively.
- **A phase-offset pipelined dataflow** that orders wave slots to maximize inter-device slack and overlaps boundary exchange with interior computation whenever the link satisfies the latency-transparency criterion.
- **Evaluation across three substrates**: three classes of GPUs, 1-D and 2-D Xilinx FPGA meshes, and a fabricated four-chip 28 nm ASIC, where the streaming schedule holds per-sweep time flat in chip count at $0.985\text{ }\mu\text{s}$ for 27,648 spins and 4.81 pJ per update, $32.4\times$ faster than an H200 and three orders of magnitude lower in energy per spin update on the same Pegasus topology.

II. BACKGROUND

A. Motivation

Ising machines cast computation as the evolution of a dynamical system. Instead of executing a general sequence of instructions, the hardware maintains a network of interacting binary variables and lets their collective dynamics search for low-energy states. This formulation makes the Ising model especially attractive for acceleration: once an optimization problem is expressed as a spin system, the same local interactions that define the energy function also define the structure of the computation. The Ising model, originally formulated to describe ferromagnetism in statistical mechanics [5], provides a computational framework for a broad class of combinatorial optimization problems. The system energy is:

$$E(m) = -\sum_{i < j}^N J_{ij} m_i m_j - \sum_i^N h_i m_i \quad (1)$$

where N is the number of spins, $m_i \in \{-1, +1\}$ are binary spin variables, J_{ij} are pairwise interaction couplings, h_i are external fields, and $m = [m_1, \dots, m_N]^T$ denotes the spin configuration. The ground state of E encodes the solution to

the optimization problem. The couplings define a graph of interconnections.

Lucas [1] has shown that all 21 of Karp's original NP-complete problems [16] can be mapped to Ising formulations. However, most of these formulations lead to dense connectivity, whereas practical Ising hardware typically exposes fixed sparse connectivity. Sparsification techniques, either algorithmic [17] or based on directly reducing edge densities [18], [19], are therefore essential for bringing these mappings into the high-parallelism, on-chip regime needed for efficient hardware implementation.

B. Gibbs Sampling

To sample low-energy configurations, we target the Boltzmann distribution, which concentrates on ground states at thermal equilibrium. We sample it with Markov Chain Monte Carlo (MCMC) methods such as Gibbs sampling [20], which iteratively updates individual spins conditioned on their neighbors.

For spin i , the update rule for a bipolar Ising problem is:

$$I_i = \sum_{j \in \mathcal{N}(i)} J_{ij} m_j + h_i, \quad (2)$$

$$m_i \leftarrow \text{sgn} [\tanh(\beta I_i) - u], \quad (3)$$

where $\mathcal{N}(i)$ is the set of neighbors of spin i ; I_i is the *local field*, and $u \sim \text{Uniform}(-1, 1)$ is a random sample. Eq. (2) computes the weighted sum of neighbor states, and Eq. (3) applies a stochastic sigmoid activation to it.

Here $\beta = 1/T$ is the inverse temperature, which controls the sharpness of the stochastic update. At high temperature (small β), updates are nearly random and the chain explores the configuration space freely; at low temperature (large β), updates are driven toward the lowest-energy states. Since the Boltzmann distribution concentrates on ground states only as $T \rightarrow 0$, sampling typically begins at high temperature to escape local minima and gradually cools (β is raised) until the system settles into a ground state. This process is called *simulated annealing* [21].

The computational cost per spin update is $O(d_i)$, where $d_i = |\mathcal{N}(i)|$ is the degree of spin i . A *sweep* is defined as N attempted spin updates (one per spin). Formally, the spins form a coupling graph (CG) with N spins and M edges. For the

regular graphs benchmarked in this paper the degree is uniform across spins, giving an average degree $\bar{d} = d_i = 2M/N$.

C. Time-to-Solution

Because the update is stochastic, a single run may or may not reach a good state, so quality is reported as time-to-solution (TTS): the expected wall-clock time to reach a target-quality solution with probability at least p_{target} (99% in this work) [22]–[25]. A run of duration t_f succeeds with probability p_{succ} ; repeating independently for R iterations succeeds with probability $1 - (1 - p_{\text{succ}})^R$, so requiring at least p_{target} gives $R = \ln(1 - p_{\text{target}}) / \ln(1 - p_{\text{succ}})$ iterations of cost t_f each:

$$\text{TTS}(t_f) = t_f \cdot \frac{\ln(1 - p_{\text{target}})}{\ln(1 - p_{\text{succ}})}. \quad (4)$$

t_f and p_{succ} trade off against each other, discussed further in Section V.

D. Update Schedules, Sparsification, and Graph Topology

Gibbs sampling is sequential by construction: each spin update must observe the most recent values of its neighbors to preserve detailed balance, and naively updating all spins in parallel violates this condition [26]. Fully asynchronous updates, in which each spin fires at a random instant, relax this requirement and are natural for stochastic devices [27]–[29]. The induced statistical error is theoretically bounded, and for sparse problems the iteration count grows by only a factor of $1 + O(1/N)$ [30], making asynchrony attractive when correctness can be traded for throughput.

The principled middle ground is colored Gibbs sampling, which separates spins into χ independent sets such that no two spins of the same color share an edge [28], [31]. All spins of one color update simultaneously, and the effective parallelism is N/χ per color step. The chromatic number $\chi(G)$ thus becomes the central architectural parameter: minimizing χ maximizes the parallel speedup that any platform can extract from a colored update schedule.

Because χ is set by graph topology, the choice of graph dictates both parallelism and memory behavior. Dense all-to-all graphs are the worst case on both axes: their $O(N^2)$ coupling matrix exhausts on-chip memory at moderate N , and because $\chi = N$ the parallelism collapses to $O(1)$, leaving barrier overhead as the dominant cost. Sparse regular graphs invert both problems. The 8-connected kings graph (Fig. 1 right panel) and 15-connected Pegasus graph serve as running examples throughout this paper; both have $O(N)$ memory footprint, $\chi = 4$, and predictable high-locality access patterns, which suits them to both GPU coalescing and FPGA on-chip routing [32].

Although the experiments use the regular graphs as controlled benchmarks, the execution model is not limited to regular graphs. Irregular sparse graphs can also be colored and updated in parallel, and many require only a small number of colors [28]. Dense mapped instances are a separate regime

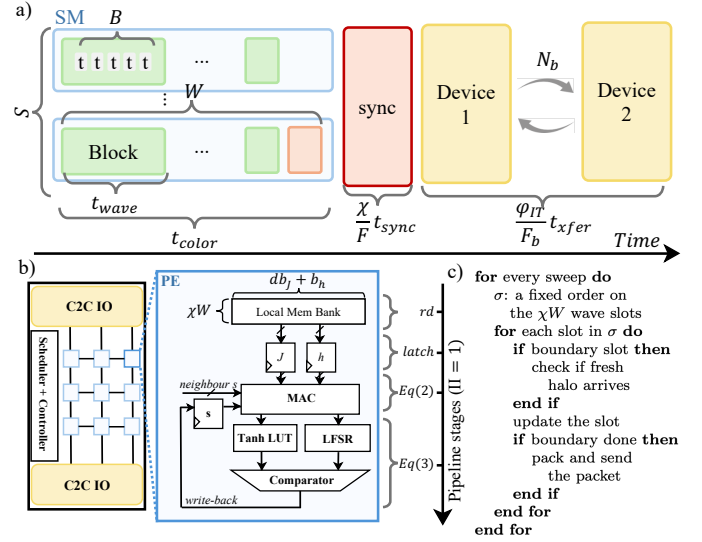


Fig. 2. (a) One sweep as Eq. (5) reads it. A straggling block marked in orange. (b) The SP architecture: a processing-element (PE) array under one scheduler, a local memory bank per PE, chip-to-chip (C2C) IO on the edges. The update algorithm is shown in pseudocode.

known to scale poorly in digital Ising hardware [19]. This paper therefore focuses on sparse scalable hardware graphs.

Real-world optimization problems frequently map to dense graphs, so sparsification [28] is what makes hardware-efficient execution possible. Replacing an all-to-all instance with a structured sparse embedding shrinks the working set and reduces the number of floating point operations per spin.

III. MODELING THE BOTTLENECK

Each Ising spin update moves a fixed set of operands through Eqs. (2) and (3). The machine fetches couplings J and bias h , accumulates the local field, draws a random sample, and writes the new spin state. An example datapath is shown in Fig. 2b. A sweep applies this update once to every spin, so N updates complete one sweep. We decompose sweep time into compute, memory, synchronization, and inter-device communication costs, then use this decomposition to identify bottlenecks on dynamically scheduled GPUs and statically scheduled streaming platforms (SPs) like FPGA and ASIC. For GPUs, we instantiate the model on a CUDA kernel [33] and calibrate and validate each term across the Ada RTX 4070, Blackwell RTX 6000, and Hopper H200. Since statically scheduled platforms are deterministic, they can be modeled from synthesis-time constants alone, with no empirical fit.

A. The Sweep-Time Decomposition

We compare platforms using spin-update throughput, $\Theta = N/t_{\text{sweep}}$, measured in attempted updates, or flips per second (Eq. (3)). This schedule-dependent metric remains comparable across platforms even though the accepted-flip rate falls at low temperature. As Fig. 2a illustrates, we decompose the time per sweep as:

$$t_{\text{sweep}} = \chi \cdot W \cdot t_{\text{wave}} + \gamma \cdot t_{\text{sync}} + t_{\text{comm}}. \quad (5)$$

Here, χ is the chromatic number of the CG; W is the number of serialized waves per color step (Section III-B); γ is the number of synchronization points per sweep; t_{sync} is the cost of each synchronization within a device, and t_{comm} is the inter-device communication time (Section V). This additive form follows the BSP [34] and LogP [35] practice of separating compute, communication, and barrier costs.

Following the roofline model [36], we bound each wave by compute or memory bandwidth: $t_{\text{wave}} = \max(t_{\text{compute}}, t_{\text{mem}})$ (Sections III-C and III-E). On a GPU the coupling fetch stalls the wave (Section III-E2). On an SP the fetch occupies its own pipeline stage, so t_{mem} overlaps t_{compute} and the initiation interval (II) determines the performance (Section III-E3).

Closed forms for each term in Eq. (5) let us compute platform crossovers from the workload parameters ($N, \bar{d}, \chi, K$) and a few measured hardware constants, using the released model [33], rather than discovering those crossovers by experiment.

B. Wave Quantization

Finite execution width quantizes each color step into one or more serialized waves. Each platform updates spins with S_{compute} units in parallel, using SMs on a GPU and PEs on an FPGA or ASIC. GPUs dynamically dispatch blocks of B threads to SMs (Fig. 2a), whereas an SP statically assigns spins to PEs and has $B = 1$. Because exact Gibbs sampling requires a global barrier between color steps, one wave can update at most $S \cdot B$ spins per color step. Thus, $N_{\text{crit}} = \chi B S$ marks the largest problem with no wave serialization. Beyond this threshold, the N/χ updates in each color step require $W = \lceil N/N_{\text{crit}} \rceil$ waves, so $t_{\text{color}} = W \cdot t_{\text{wave}}$. Even one block beyond a multiple of S adds a full t_{wave} while the remaining units wait (Fig. 2a). This wave quantization is equivalent to inflating the chromatic number from χ to χW . This motivates multi-device scaling. We define a slot as one (color, wave) pair, and all the χW slots in a sweep are scheduled with order σ (Fig. 2b). On a GPU, dynamic block dispatch provides no within-grid ordering guarantee, so global barriers separate colors and enforce a color-major σ . On an FPGA or ASIC, compile-time dependency resolution permits any legal ordering of the χW slots, which helps hide the I/O latency (Section V).

C. The Compute Term and Arithmetic Intensity

Each spin update is a sparse dot product of d multiply-accumulates, a bias add, and one spin write (Eq. 3): $2d + 1$ FLOPs plus a degree-independent overhead of one sigmoid and six integer operations since we use xorshift32 [37]; Linear-feedback shift register (LFSR) class pseudorandom number generators (PRNGs) suffice for optimization [28], [38] while learning may need xoshiro or Philox [38]–[40]. On SPs the generator is more an architectural than a quality choice. On a 26-spin fully-connected FPGA array, four PRNG configurations and an external true random number generator (TRNG) land within 0.49 points of ground-state success. If the LFSR’s repeating period is small, or if one LFSR is shared between

PEs, the stream repeats or there could be correlated sequential reuse and the solution quality will degrade. Also notice that a 1-bit spin state ($s_i \in \{0, 1\}$) turns each multiply into a conditional add.

We present the first roofline analysis of Ising workloads. Ising sweeps sit below the roofline ridge and are memory-bandwidth-limited for both sparse and dense graphs. The relevant arithmetic intensity is $\text{AI} = (2d+1)/[d(b_J+b_s)+b_{\text{fix}}]$, with b_J and b_s the coupling and spin-state widths in bits and b_{fix} the fixed per-update traffic (the bias read and the spin write-back, $b_h + b_s$). For moderate d , AI approaches $2/(b_J + b_s)$, and even at minimal precision $\text{AI} = 1.8$ FLOP/byte. Operations and traffic both scale with d , so sparse and dense graphs have similar AI. This metric is less predictive for SP implementations, where distinct pipeline stages execute per-spin operations concurrently and the initiation interval sets the limiting throughput (Section III-E3) [41].

The kernel is memory-bound at every measured operating point; on the RTX 4070, ALU utilization reaches only 22% at $d = 16$. Its compute work is structurally unbalanced across $O(d)$ floating-point accumulations, one sigmoid on the special-function unit, and a small number of integer operations for random-number generation and addressing. Ada and Hopper execute these three pipelines concurrently, whereas Blackwell time-multiplexes them on a unified datapath.

D. The Barrier Cost

The barrier term γt_{sync} and the exchange term t_{comm} in Eq. 5 can be interpreted as one coordination cost at both within-device and across-device scales. Colored Gibbs sampling forces a global barrier between color classes so each spin sees committed neighbor values. On GPUs, μs -scale synchronization makes t_{sync} significant at small B (Fig. 3a). We parameterize synchronization frequency by the *synchronization interval* F : the number of color steps between global barriers, giving $\gamma = \chi/F$ synchronization points per sweep. $F = 1$ gives exact Gibbs where we synchronize after every color step while $1 < F < \infty$ gives bounded staleness discussed in Section V-B.

Raising F shrinks the barrier term γt_{sync} but degrades success probability $p_{\text{succ}}(\gamma)$, since spins observe increasingly stale neighbor states; the opposing effects produce a concave TTS speedup curve shown in Fig. 6.

GPU barrier cost was measured by injecting redundant barriers into the cooperative kernel; the slope of sweep time versus barrier count gives t_{sync} (Fig. 3a). The barrier cost is a function of participating blocks, and RTX 6000 Blackwell has the largest t_{sync} because its cooperative grids are largest. Barriers are also not pure overhead: removing them at $N = 2 \times 10^6$ on the RTX 4070 makes the run 42% slower, since threads drift out of lockstep and lose streaming locality on a bandwidth-bound device, as illustrated by the grid-striding transition in Fig. 3a. Thus, the barrier acts as a cache-locality synchronizer beneficial to bounded staleness discussed in Section V.

On SPs, whose pipeline-control barriers propagate in nanoseconds, $t_{\text{sync}} = \mathcal{O}(\text{ns})$; compile-time work assignment gives deterministic timing. Color-step synchronization comes from pipeline control where the only cost is the *pipeline drain*, bounding in-flight writes of the finishing color before the next color reads them.

E. The Memory Term

On a platform that fetches couplings through a memory hierarchy, compute is not the limiting factor. The wave time is therefore the memory service time, and t_{mem} sets single-device throughput. Layout and residency tier are design choices (Section IV); here we model the traffic each update moves and the service time it implies.

1) *Memory traffic*: The memory is modeled with a hierarchy-traversal service time and a residency-weighted, occupancy-dependent service rate rather than the fixed peak bandwidth that roofline bounds by. The working set picks the bandwidth tier; the traffic sets how long that tier is occupied, and traffic always exceeds memory footprint. Per sweep, J , indices, and h are *streaming* (one read, no reuse), while each spin state is read on average d times; with spin cache hit rate $\rho_s \in [0, 1]$, the per-spin traffic is

$$Q_{\text{bit}} = \underbrace{d(b_{\text{idx}} + b_J)}_{\text{coupling, self-bias (streaming, no reuse)}} + \underbrace{(1 - \rho_s)d b_s}_{\text{neighbor spins (used } d \times)} + \underbrace{b_s}_{\text{spin write-back}}, \quad (6)$$

where b_{idx} is the column-index width and b_h the bias width, and the per-sweep total is $N \cdot Q_{\text{bit}}$. On SPs with private per-PE memory, $\rho_s \rightarrow 1$ by construction. On GPUs, ρ_s comes from L1/L2 hit counters. The memory traffic is streaming-dominant: zero-reuse fetches dwarf the spin terms, favoring memory-throughput-optimized architectures over cache-based ones, and under GPU LRU eviction the zero-reuse streams can even evict spin data (Section IV-B).

2) *GPU memory service time*: On GPUs, the low arithmetic intensity reduces t_{wave} to the memory service time:

$$t_{\text{wave}} = \frac{S \cdot B \cdot Q_{\text{bit}}}{\eta_{\text{mem}} \cdot \text{BW}_{\text{max}}}, \quad (7)$$

$$\eta_{\text{mem}}(x) = \frac{\text{BW}_{\text{achieved}}(x)}{\text{BW}_{\text{max}}} = \eta_{\text{max}} \cdot f(x),$$

where $B \cdot Q_{\text{bit}}$ is one block's traffic, BW_{max} the effective device bandwidth (a three-level harmonic blend of measured L1/L2/DRAM bandwidths weighted by hit rate), and $\eta_{\text{mem}} \in (0, 1]$ the fraction of that bandwidth sustained, below 1 when memory-level parallelism is too low to saturate the pipeline. $f(x)$ is a saturating hyperbola as a function of the resident-thread pool x : by Little's Law [42], delivered bandwidth grows linearly with the requests in flight and levels off at the ceiling once enough threads are resident to cover the load latency [43].

3) *Statically scheduled platforms*: In SPs, t_{wave} in Eq. (5) is set by the pipeline's *initiation interval* Π , giving $\Theta = Sf/\Pi$ and

$$t_{\text{sweep}} = \frac{N \cdot \Pi}{S \cdot f} + \gamma \cdot t_{\text{drain}}, \quad (8)$$

where Π is fixed at synthesis by the most-contended resource (DSP block, memory port, or write-back path) and f the clock frequency. Unlike a GPU's fixed-in-silicon S , an SP sets S at synthesis, coupling the free parameters: added PEs draw on a shared budget of memory ports, LUTs, and annealing DSPs, and past a point exhaust it or force a lower f or higher Π to close timing.

4) *The throughput ceiling*: On SPs the unified model collapses to a closed-form ceiling on Θ . With proper pipelining, $t_{\text{drain}} \rightarrow 0$ (Section III-D) and $\eta_{\text{ovlp}} \rightarrow 1$ so $t_{\text{comm}} \rightarrow 0$ (Section V-A), giving $t_{\text{sweep}} = \chi W t_{\text{wave}}$. Expanding W and t_{wave} at large N ($\lceil N/N_{\text{crit}} \rceil \cdot N_{\text{crit}} \approx 1$),

$$\Theta_{\text{SP}} = \frac{N}{t_{\text{sweep}}} = \min \left(\underbrace{\frac{S \cdot f}{\Pi}}_{\text{compute ceiling (on-chip)}}, \underbrace{\frac{\eta_{\text{mem}} \text{BW}_{\text{max}}}{Q_{\text{bit}}}}_{\text{bandwidth ceiling (off-chip)}} \right). \quad (9)$$

The terms are equal when the working set is on-chip, since per-PE memory is provisioned to feed the pipeline; the residency threshold (Eq. (10)) marks the cliff between regimes, and both ceilings are *independent of N* – within a regime the streaming machine delivers constant throughput and per-sweep time exactly linear in N . The same reduction for the GPU (Eq. (7)) sends $\Theta_{\text{GPU}} \rightarrow \eta_{\text{mem}} \text{BW}_{\text{max}}/Q_{\text{bit}}$ as $N \rightarrow \infty$, the *same* bandwidth ceiling as Θ_{SP} , reached only after amortizing the barrier tax and climbing the occupancy ramp $\eta_{\text{mem}} < 1$. At large N the levers are BW_{max} , η_{mem} , and Q_{bit} – why the degree cancellation of Section III-C does not imply degree-independent throughput.

F. Model Calibration

Block size B packages the N/χ spins per color step into blocks across the S SMs, trading occupancy against wave quantization. The grid-wide barrier requires cooperative launch, which caps the grid at the hardware's resident-thread limit; past N_{crit} each thread loops over multiple spins (a grid-stride loop), and t_{sweep} jumps 50% in one step (Figure 3a). We fixed $B=256$ for calibration, which predicts held-out sweeps (Fig. 3b) to 2.5–10.7% mean absolute percentage error (MAPE), mean 4.8% over 18 (GPU, block) configurations for $B=64$ –1024.

The model takes architectural constants from the datasheet (S , DRAM bandwidth, cache capacity) and a few empirical parameters measured once per (GPU, kernel) pair [33]: cache hit rates from `ncu` feed the harmonic blend for BW_{max} , a t_{sync} microbenchmark (Section III-D) supplies the barrier term, and a least-squares fit of the $B = 256$ sweep gives the three constants of $\eta_{\text{mem}}(x)$. Across the full sweep ($N=100$ to 2×10^6) on the RTX 4070, RTX 6000, and H200, the calibrated model reaches MAPE 5.2%/2.6%/3.3% at the calibration block size and 7.2%/4.1%/3.8% averaged over five held-out block sizes, as summarized in Fig. 3b. Refitting on half the

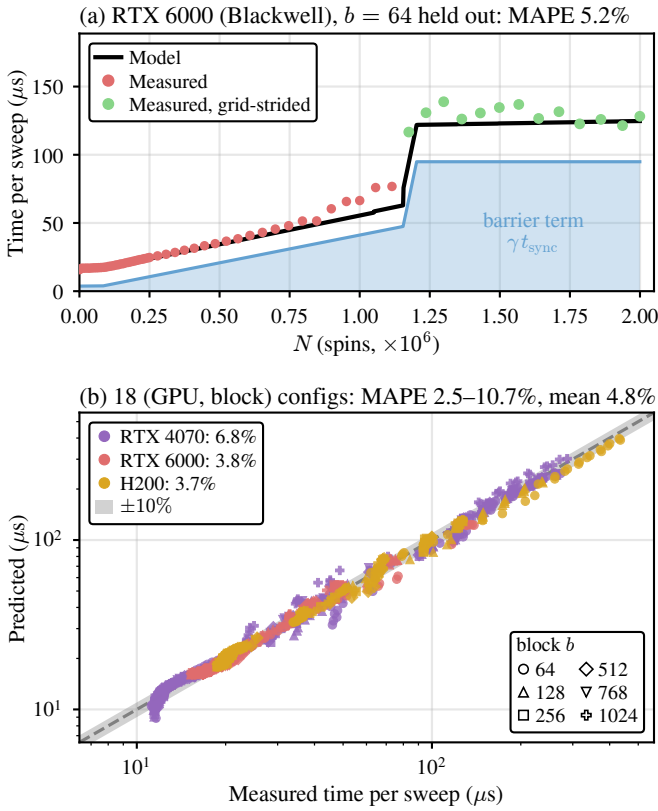


Fig. 3. One calibration per GPU (the $B=256$ curve and a barrier microbenchmark) predicts sweep time at held-out block sizes with no refitting. (a) Exemplar sweep on the RTX 6000 at a block size the calibration never sees, with the barrier term shaded and the grid-striding sizes marked. (b) Model against measurement for all 18 (GPU, block) configurations, 63 sizes each; the band is $\pm 10\%$.

problem sizes and testing on the disjoint half moves error by at most one point on every GPU, so platform dependence is confined to a few physically grounded constants, not the decomposition’s form of Eq. (5); supplying the exact measured $\eta_{mem}(x)$ in place of the three-constant fit tightens the match to $MAPE < 2\%$ on all three GPUs, proving the residual is the price of summarizing η_{mem} in closed form.

With the performance bottlenecks localized, Section IV characterizes the low-AI workload and the cost of working-set spill. Sections V and VI then present the two system-level acceleration approaches: bounded-staleness sampling and phase-offset pipelining.

IV. DATA PLACEMENT: t_{mem}

Working-set spill changes the dominant cost. On a cache-resident GPU at $N=10^4$, t_{mem} and γt_{sync} account for 79% and 20% of sweep time, respectively. After the working set spills at $N=10^6$, the memory share rises to 96%. On the ZCU106, $t_{compute}$ accounts for 92% with J in BRAM and 77% when streamed from DDR. This section measures the spill-induced performance loss and the format, precision, and placement choices that determine its crossover.

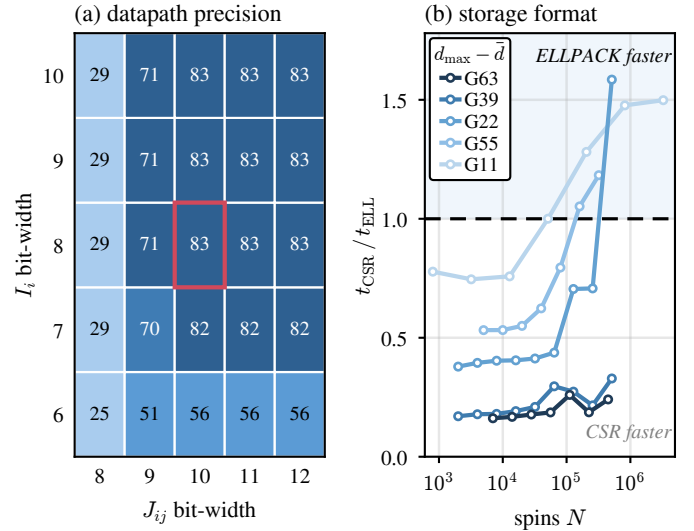


Fig. 4. Coupling precision and storage format trade-offs. (a) Ground-state success rate (%) on a 26-spin, fully connected array as a function of integer field width and stored coupling width (averaged over 100 Sherrington-Kirkpatrick instances, evaluated at 3 fractional bits). The boxed cell highlights the optimal design point. (b) Execution time ratio (t_{CSR}/t_{ELL}) on an NVIDIA RTX 4070 against spin count N , five G-set instances block-replicated from their published sizes, ordered by irregularity. The three near-regular instances favor ELLPACK as N grows.

A. Storage Format and Footprint

The working set is the coupling matrix J , the bias vector h , and the spin state. Dense storage grows as N^2 and loses to sparse storage once $\bar{d}$ falls. Of the two well-studied sparse formats [44], [45], compressed sparse row (CSR) stores a row pointer per spin and packs each coupling list tightly, while ELLPACK [46] drops the pointers and pads every row to d_{max} . ELLPACK beats CSR when $d_{max} - \bar{d} < b_{ptr}/(b_{idx} + b_J) = 0.67$ at our 32-bit pointers/indices and 16-bit couplings with b_{ptr} the row-pointer width. Thus, ELLPACK only wins on near-uniform-degree graphs. Kings and Pegasus are both regular, so ELLPACK was used in our study.

On SPs the static schedule makes pointers and indices implicit: only couplings and bias are stored, in fixed-point $Qm.n$ format ($b_J = m + n + 1$ bits with sign). Topologies having higher degree like Pegasus ($d = 15$ [9], [47]) or annealing schedule with larger β need extra integer bits for the larger local field I_i (Eq 2). Fig. 4a sweeps the two widths independently. By choosing the optimal precision, we can achieve a geometric-mean $3.4\times$ TTS reduction at an identical energy target.

The measurements identify on-chip working-set residency as the main performance crossover for this low-AI workload. For ELLPACK on a regular graph, residency requires

$$N \cdot \bar{d} < \frac{C_{on-chip}}{b_{idx} + b_J}, \quad (10)$$

where $C_{on-chip}$ is the on-chip cache capacity. A mapping meets the *on-chip residency criterion* when Eq. (10) holds.

B. GPU Memory Measurement

A GPU cannot satisfy the on-chip residency criterion at the sizes that motivate this work. On the RTX 4070 (128 KB L1, 36 MB L2), effective bandwidth drops $2.9\times$ once t_{mem} contributes 96% of the sweep past the crossover of Eq. (10). On kings graphs, couplings, indices, and biases carry $\sim 75\%$ of traffic with zero reuse, so marking those loads evict-first avoids cache pollution and yields $1.44\times$ beyond L2. Scattered neighbor access of every spin keeps L1 hit rates graph-dependent, $\sim 40\%$ for kings, 13% for Pegasus, since kings has a better spatial locality. The near-memory SPs never pay this price of sparse gather [48]–[50].

CSR walks one spin’s coupling list per thread; ELLPACK’s column-major layout lets a warp read 32 consecutive addresses at the cost of an index multiply-add. CSR’s cheaper addressing wins while the kernel is memory-latency-bound; once memory-bandwidth-bound, ELLPACK’s coalesced loads and smaller footprint win by up to $1.6\times$ (Fig. 4b). The format choice is therefore decided by graph size, degree and irregularity.

C. Streaming Datapath

A statically scheduled SP prevents working-set spill by holding the couplings on chip and feeding every PE from a private local memory. We implement the streaming architecture on a Xilinx ZCU106 and XEM8320 with both kings and Pegasus graphs in Q6.3 fixed-point, running at 100 MHz. With kings/Pegasus graph, every spin’s eight/fifteen couplings and bias form a 90-bit/160-bit BRAM word; updates flow through a three-stage pipeline at $\text{II} = 1$: issue memory read, latch the memory output and sample the new state. With $t_{\text{drain}} = 0$, $t_{\text{sync}} = 0$, and each PE time-multiplexes into waves once $N/\chi > S$, utilizing χW rows of memory. DSP48E2 pre-multipliers were used to implement annealing.

As N grows, the working set spills through BRAM, LUT-RAM, URAM, and off-chip DRAM (Fig. 10, FPGA curves); a prefetch engine on the DRAM tier cuts the per-spin update time from ~ 335 to ~ 3.5 ns. Unlike the GPU model, this streaming model needs no empirical fitting, since its inputs are synthesis-time and datasheet constants and it tracks measured throughput to within a few percent across all four memory tiers.

V. BOUNDED-STALENESS SAMPLING

This section presents two bounded-staleness mechanisms that reduce synchronization frequency. We use one bound, $F \geq 1$, for both. A consumer may observe a neighbor state that persists for up to F update intervals, and $F = 1$ is the fully synchronized baseline. Let E_{bi} denote boundary–interior interactions within a partition and E_{cross} denote interactions crossing a partition boundary. Multi-rate bounded staleness applies F to E_{bi} ; cross-partition bounded staleness applies the same bound to E_{cross} . The next section presents phase-offset pipelining, the second system-level acceleration approach. We evaluate the approaches on H200 pairs and Xilinx FPGAs.

A. Graph Partitioning

A single device has only t_{sync} and no t_{comm} . As the working set grows, the sweep time on one device becomes marginally more expensive. When the sweep time reduction beats the marginal communication cost, partitioning becomes beneficial (Fig. 8). The coupling graph (CG) is partitioned across K devices physically connected with an *interconnect topology* (IT). When the IT matches the CG structure, every boundary exchange is single-hop: logical nearest-neighbor couplings become physical links. After partitioning, the per-device spin count becomes N/K and the working set may fall into the faster memory tier. Also the per-device wave $W(K)$ drops from $\lceil N/N_{\text{crit}} \rceil$ to $\lceil N/(KN_{\text{crit}}) \rceil$, but the couplings crossing a partition boundary require periodic state exchange.

Let N_b be the per-device boundary volume exchanged at each boundary synchronization point, which need not coincide with interior barriers. A balanced 2D partition of a geometrically local, bounded-degree graph (kings shown in Fig. 5) puts cut edges on the perimeter, $N_b = O(\sqrt{N/K})$, so the boundary-to-interior ratio vanishes as N grows. The per-sweep communication time is defined as:

$$t_{\text{comm}} = \varphi_{\text{IT}} t_{\text{xfer}} (1 - \eta_{\text{ovlp}}), t_{\text{xfer}} = t_{\text{lat}} + \frac{N_b b_s}{\text{BW}_{\text{link}}}, \quad (11)$$

where t_{xfer} splits into a fixed latency term and a bandwidth-limited payload [35]; η_{ovlp} is the fraction of t_{xfer} hidden behind computation; φ_{IT} is the chromatic number of IT: the network-level counterpart of χ on the CG.

Boundary spins may be exchanged below the interior barrier rate γ of Section III-D. By the same construction, a sweep performs $\gamma_b = \varphi_{\text{IT}}/F$ exchanges at the boundary, where the common bound F applies to E_{cross} for cross-partition bounded staleness. Each sweep has φ_{IT} possible exchange points. We do not model γ_b explicitly in the t_{comm} equation; its effect is captured by η_{ovlp} .

B. Algorithm

Every device runs χW wave slots per sweep (Section III-B). Fig. 5a shows an 8-spin kings graph example with each device hosting 4 spins and 1 PE. In the serialized baseline, both transfers within a sweep lie on the critical path ($\eta_{\text{ovlp}} = 0$, $\gamma_b = 1$). We also observe that a half-duplex link suffices.

The “IDLE” periods in the serialized schedule motivate two bounded-staleness mechanisms. Multi-rate bounded staleness lets interior p-bits continue updating while propagation from cut-facing boundary p-bits is delayed. Cross-partition bounded staleness lets a boundary p-bit update before a fresh neighboring-partition halo arrives. Figure 5 shows both mechanisms.

Under multi-rate bounded staleness, an interior p-bit can observe a stale boundary-neighbor state on E_{bi} . Boundary and interior p-bits refresh at throughputs Θ_b and Θ_{interior} , with $\Theta_b b_s \leq \text{BW}_{\text{link}}$. The common bound is $F = \Theta_{\text{interior}}/\Theta_b$, the number of interior updates over which a boundary state

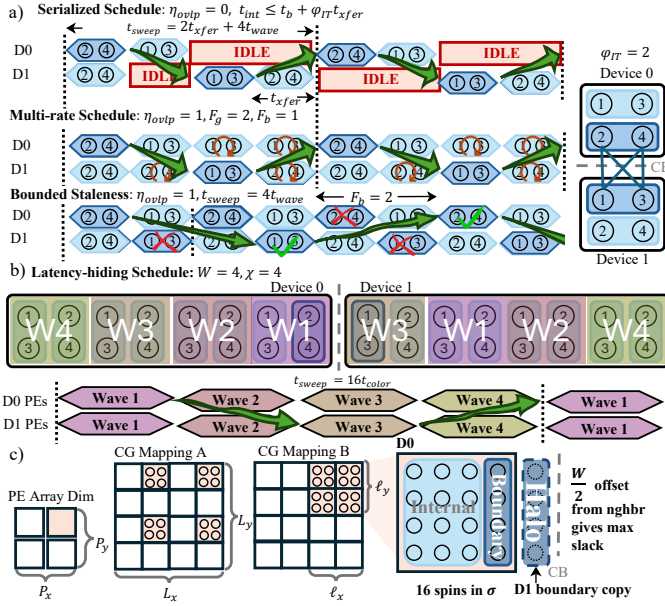


Fig. 5. Boundary exchange across a two-device cut (boundary p-bits marked in dark blue). (a) Timelines for the serialized baseline, multi-rate bounded staleness, and cross-partition bounded staleness. (b) Latency-hiding timeline at $S=1$, $W=4$, $\chi=4$. (c) Mapping the PE array to the CG at $S=4$, $W=16$, $\chi=4$.

may persist. On a geometrically local graph, the interior-to-boundary ratio N_{int}/N_b grows as $\sqrt{N_{\text{dev}}}$, where $N_{\text{int}} = N_{\text{dev}} - N_b$ and $N_{\text{dev}} = N/K$. Most update work can therefore continue at the interior rate when the link limits boundary refresh.

Under cross-partition bounded staleness, a boundary p-bit observes a stale neighbor state on E_{cross} and samples from a conditional computed with an aged halo. The same F bounds the number of update intervals for which that propagated state may persist. Raising F reduces boundary exchange demand even when $\eta_{\text{ovlp}} = 1$ (Fig. 5a).

Both mechanisms permit stale neighbor observations and therefore depart from the fully synchronized Gibbs baseline. Their measured effects differ because the stale reads occur on different edge sets. For multi-rate bounded staleness at $F=16$, the boundary p-bits' integrated autocorrelation time τ rises about $9\times$ over the $F=1$ baseline. The measured global τ is unchanged for boundary fractions N_b/N_{dev} up to 6%, rises $3.5\times$ at 25%, and slows the machine as a whole when the boundary fraction reaches 100% (Fig. 6b). Cross-partition bounded staleness directly perturbs boundary conditionals, so its solution quality must be evaluated with its throughput gain [51].

C. Evaluation

Figure 6 compares multi-rate bounded staleness, cross-partition bounded staleness, and the single-device baseline across three benchmark families: MaxCut (Fig. 1), frustrated loops, and tile planting [52], [53]. We isolate multi-rate bounded staleness by delaying boundary-state propagation to interior p-bits and use the number of interior updates per

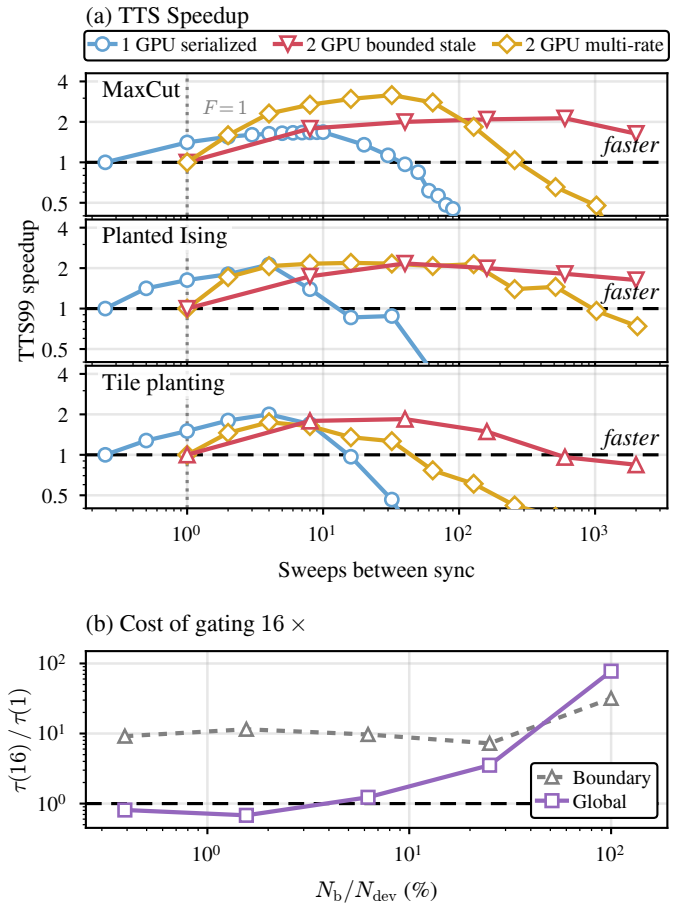


Fig. 6. (a) TTS99 speedup against the common staleness bound F for three kings families at $N=10^4$; the tile-planting two-device arm is the open-boundary sibling. (b) Mixing-time cost when multi-rate bounded staleness lets a boundary state persist for $F=16$ interior update intervals.

boundary refresh as F . We evaluate the throughput-quality tradeoff by sweeping the same bound for each applicable edge set. As synchronization becomes less frequent, t_{sweep} and p_{succ} both fall monotonically. TTS reaches its optimum where the sweep-time gain no longer compensates for the success-probability loss.

The blue curve varies F on interactions within one device. The yellow and red curves apply F to E_{cross} and E_{bi} , respectively. Within a single GPU, reducing synchronization frequency gains up to $2\times$ but degrades sharply beyond the optimum because every coupling is affected and the barrier also structures cache locality (Section III). Across devices, the two bounded-staleness mechanisms yield up to $2.95\times$. Only p-bits incident on the selected boundary edges observe stale neighbors, so the useful range extends to larger F .

Graph degree and density set the useful range of F . Sparse graphs often tolerate moderate staleness, while dense graphs require more frequent synchronization because each stale neighbor observation affects more interactions [28], [30]. We measure this effect on planted frustrated-loop instances hosted on a degree-8 kings graph ($N=4,096$) and a degree-15 Pegasus

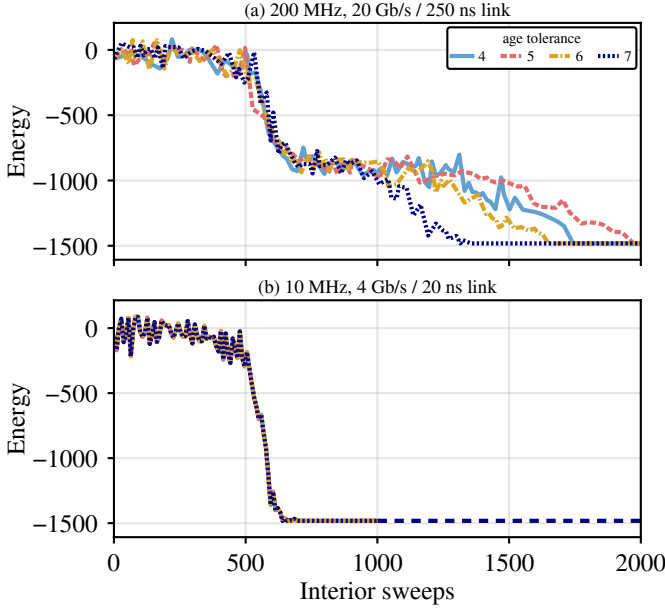


Fig. 7. Age tolerance on a 20×20 kings instance, energy against interior sweeps, under cycle-accurate simulation. (a) A 200 MHz array on a 20 Gb/s, 250 ns link. (b) A 10 MHz array on a 4 Gb/s, 20 ns link. Each tolerance carries its own dash pattern. In (b) the four coincide exactly, so they overprint as a single interleaved multicolour stroke: at this link latency the boundary never lags and the tolerance is never reached. That run ends at 1,001 sweeps, and the plain navy segment beyond it carries the converged energy across the shared axis rather than reporting measurement.

graph ($N=5,640$). At $F = 64$, the residual energy above the planted optimum rises by 5% on kings and 30% on Pegasus, each relative to its own synchronous baseline. At $F = 256$, the increases are 14% and 49%, respectively.

D. Emulated Multi-Device Platform

The age-tolerance results of Fig. 7 run on an emulated machine. One large FPGA hosts several small p-bit array instances, assembled using Vitis, and the inter-device links between them are parameterized channels with a settable rate and latency. With this setup, we can emulate and sweep the array clock, the link bandwidth, and the link latency. The PE arrays themselves are the synthesized design, with the same datapath, the same RNG, and the same update order as a deployed instance.

A planted MaxCut instance with a 20×20 king’s graph was partitioned across four emulated devices. The multi-rate bounded-staleness mechanism is evaluated on this emulation platform. Each p-bit carries a hardware *age*, measured in emulated clock cycles since a local update or link arrival. A consumer stalls at the configured age tolerance. We normalize that hardware tolerance to the common bound F in p-bit update intervals when comparing the two mechanisms. Plotting energy against interior PE sweeps (Fig. 7) normalizes out the interior rate, so the curves separate only when the boundary lags. Over a low-latency link the boundary keeps up and every age tolerance traces one curve; over a high-latency link the sweep budget to reach the ground state spreads from

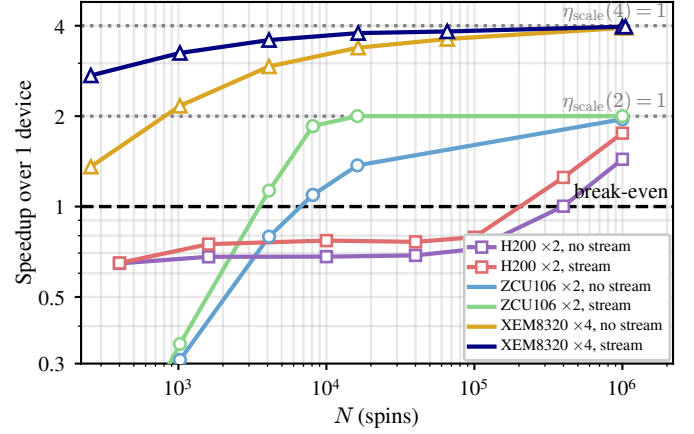


Fig. 8. Two-device speedup and streaming speedup vs. problem size on two H200, two ZCU106 and four XEM8320. Ideal scaling efficiency for 2 devices and 4 devices marked in grey dashed lines.

~ 1300 to ~ 2000 sweeps with no monotone ordering in the tolerance. Where the link cannot feed the boundary, multi-rate bounded staleness recovers part of the sweep budget a high-latency link would otherwise cost. On larger instances, too loose of tolerances settle above the ground energy, so the common bound F should remain finite, consistent with Fig. 6a.

VI. PHASE-OFFSET STREAMING: HIDING t_{comm}

Multi-rate bounded staleness and cross-partition bounded staleness pay off only while the spin count stays small. At small N the sweep is short and the fixed cost of a boundary refresh takes a large share of it, so lowering the refresh rate buys real wall-clock time; at large N the interior updates amortize that cost (N_b/N vanishes, Section V-A) and the share left to recover shrinks toward zero, while the quality penalty of stale reads remains. What survives at scale is the transfer time t_{xfer} itself. Phase-offset pipelining overlaps that transfer with interior computation rather than reducing its frequency.

A. Algorithm and Benefits

A sweep has χW slots (Section III-B) spread across φ_{IT} phases, and we assume the CG matches the IT graph.

Each direction in a pair of devices in a maximal IT clique carries its own *directed dependency*: D_i produces its boundary spins, D_j consumes them. The slack of the dependency $D_i \rightarrow D_j$ is the number of wave slots between D_i ’s last producing slot and D_j ’s first consuming slot. The φ_{IT} devices of a maximal IT clique cut a cycle of χW wave slots into φ_{IT} gaps summing to χW , one gap of slack per directed dependency. The phase-offset streaming algorithm designs the slot order σ to maximize the minimum slack, which forces them equal at $\chi W / \varphi_{\text{IT}}$ slots (the half-sweep offset at $\varphi_{\text{IT}} = 2$).

A schedule is *latency-transparent* when:

$$t_{\text{xfer}} \leq \frac{t_{\text{sweep}}}{\varphi_{\text{IT}}} - t_{\text{stage}}, \quad (12)$$

where t_{stage} is the staging time, from the start of a boundary spin's update to the moment its packet is assembled and ready to send over the link. t_{stage} should therefore be as small as possible, motivating smaller packets dispatched as soon as they are computed. If this equation's condition is satisfied, the transfer completes inside the latency-hiding window driving $\eta_{\text{ovlp}} = 1$. The transfer still occurs, but it is fully overlapped and no longer lengthens the sweep in Eq. (5).

Expanding t_{xfer} using Eq. 11 and writing $N_{\text{dev}} = N/K$ and $\Theta_1 = N_{\text{dev}}/t_{\text{sweep}}$ (Θ_K being the K -device array rate) gives:

$$N \geq \varphi_{\text{IT}} \Theta_K t_{\text{lat}}, \quad \text{BW}_{\text{link}} \geq \frac{\varphi_{\text{IT}} \Theta_K N_b b_s}{N}. \quad (13)$$

The left equation shows the bandwidth-delay product in spins: a device must hold enough interior work to cover its own link's round trip. Fig. 8 shows this limitation together with when splitting work across two devices can be beneficial. The limitation of this streaming algorithm is that it applies only when $W\chi > S$; the more spins, the greater the benefit (Fig. 8). **Multi-rate bounded staleness and cross-partition bounded staleness can help when $W\chi < S$.** Fig. 8 reports speedup rather than TTS: under exact Gibbs, a t_{sweep} speedup translates directly into TTS reduction via Eq. (4).

Also if we rearrange the equation with Θ_K on the left, we observe that only a larger problem or a lower-latency link buys more hideable throughput. For the equation on the right, for a geometrically local bounded-degree graph that ratio is $O(1/\sqrt{N})$, so bandwidth demand falls away as devices grow and a correctly sized link is asymptotically latency-bound. Since on an all-to-all CG every spin is a boundary spin, N cancels. Also since $\varphi_{\text{IT}} = K$, the requirement collapses to $\text{BW}_{\text{link}} \geq \Theta_K$, prohibiting scaling.

B. Mapping the PE Array onto the Partition

Inside a device the partition is nested onto the $P_x \times P_y$ PE array ($S = P_x P_y$, Fig. 5c) in one of two ways. Mapping A makes the *wave* the compact unit: each PE owns a strided sub-lattice, adjacent PEs touch adjacent spins, every PE receives identical work each wave, and the spin index map is a flat formula. Mapping B makes the *PE* the compact unit: each PE owns a contiguous patch spanning all W waves. The fraction of cross-patch couplings under Mapping B falls as $1/\sqrt{\ell_x \ell_y}$. Every such coupling is a non-local neighbor access, and the platform sets what it costs: on a cache-based GPU it is a scattered read against the memory hierarchy (the t_{mem} lever of Section IV); on an SP it is an inter-PE net or a shared memory port. Mapping B shrinks both. It also localizes the inter-device link dependency: only partition-facing PEs stall on a late halo, and **under multi-rate bounded staleness** the boundary PEs can follow a link-determined rate while interior PEs run at full speed. Mapping A wins with balanced load, cheap address generation, and coalesced access on a GPU: with matched storage layouts, Mapping A is 1.2 to 1.25 $\times$ faster across $N = 4 \times 10^6$ – 3×10^8 on H200. Latency hiding is indifferent to the mapping choice: with partition-facing waves scheduled as in Section VI-A, both mappings have the same

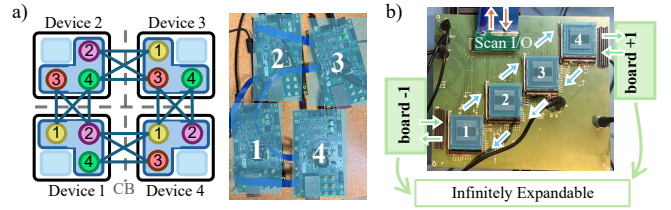


Fig. 9. (a) 4 XEM8320 connected in a 2D mesh. (b) 4 ASICs connected in a 1D mesh.

latency-hiding window. The FPGA and the fabricated chip in later sections adopt mapping A with a color-minor order.

C. 2D Mesh

At the device level, the partition shape determines how the boundary fraction scales with the number of devices. For a $\sqrt{N} \times \sqrt{N}$ lattice partitioned into K row slabs, the per-device boundary fraction is $2K/\sqrt{N}$ and reaches unity at $K = \sqrt{N}/2$. In contrast, a $\sqrt{K} \times \sqrt{K}$ block partition has boundary fraction $4\sqrt{K}/\sqrt{N}$, preserving interior locality to larger K . We therefore use a two-dimensional device mesh when scaling beyond the two-device row-slab configuration.

To demonstrate this mapping, we connect four XEM8320 boards as the 2×2 mesh shown in Fig. 9. Each board has one horizontal and one vertical nearest-neighbor link, with a measured bandwidth of 806.4 Mb/s in each direction. The board mesh matches the two-dimensional partition, so edge halos are transferred in one hop; the two diagonal corner couplings are forwarded through a neighboring board and therefore require two hops. Depending on t_{sweep} , the communication may not be fully hidden: the link latency is $t_{\text{lat}} = 116$ ns. Consequently, the transfer latency remains on the critical path for small problem sizes, resulting in a sub-optimal scaling efficiency, shown in Fig. 8.

D. Multi-Chip ASIC

The phase-offset streaming schedule is implemented on four ASICs (1.5 mm² each, TSMC 28 nm). We implement the Pegasus graph ($d = 15$ [47]), chosen for its shorter embedding chains [19], [54], at 6912 spins per chip.

Pegasus is not a planar lattice of spins, but it is a planar lattice of Pegasus *cells*: each cell holds 24 spins spanning the graph's three orientations. Therefore, all lattice dimensions in this section count cells rather than spins. Each chip maps $L_x \times L_y = 12 \times 24$ cells onto a 3×3 array of *Ising Units* (IU) ($P_x = P_y = 3$), each IU comprising 6 PEs that update one cell together, so $S = 6 P_x P_y = 54$ and the 32 cells per IU give $W = 32$ waves. All spins update in $\chi = 4$ cycles at $\text{II} = 1$; couplings and bias (Q6.3) sit in local per-IU SRAM. Since the chip has no on-chip multiplier, the host loads pre-scaled βJ and βh .

The streaming schedule moves inter-chip communication off the critical path ($\eta_{\text{ovlp}} \approx 1$): the chip's σ places the north and south boundary rows at the start of each half of the 32-wave rotation, each dispatching to its neighbor while the array

TABLE I

Approach	Control	Metric	Range
Multi-rate bounded staleness	F on E_{bi}	TTS_{99}	$1.75\text{--}3.16\times$
Cross-partition bounded staleness	F on E_{cross}	TTS_{99}	$1.84\text{--}2.17\times$
Phase-offset pipeline	latency-transparent link	t_{sweep}	$1.25\text{--}2.01\times$

works through interior spins. In the $\chi W = 128$ -slot rotation the packet dispatches as soon as its boundary wave completes. The wave itself takes 4 cycles, and the C2C interface runs at half the core clock: the 3-cycle asynchronous FIFO crossing, counted in IO-clock cycles, charges 6 core cycles, so $t_{stage} = 4 + 6 = 10$ cycles. This leaves a 54-cycle window or 415 ns of slack at the measured 130 MHz clock.

With $\eta_{ovlp} = 1$ and $t_{comm} = 0$ in Eq. (5), the schedule reaches peak throughput: for $K = 4$ ($N = 4 \cdot 6912 = 27,648$, $S = 4 \cdot 54 = 216$, 130 MHz), Eq. (8) gives $t_{sweep} = 0.985 \mu s$, equal to the single-chip sweep time, for an aggregate $\Theta_4 = 28.1$ Gflips/s (a numeric subscript on Θ counts devices). Adding chips keeps this time fixed: a K -chip machine carries $54K$ PEs alongside its $6912K$ spins, so the cycle count does not change with K . Energy is 4.81 pJ per update at 130 MHz. On one coherent 27k-node MaxCut problem across the on-PCB chip-to-chip links, with no FPGA in the loop, the solution reaches 97.11% of the global ground energy; the annealed four-chip ladder on a 27k-node instance reaches 99.909% with silicon-resident state.

VII. TOWARDS AN IDEAL ISING ACCELERATOR

Table I reports the measured effects of the two bounded-staleness mechanisms and phase-offset pipelining. Fig. 10a shows the per-sweep time each platform reaches. At $N=27,648$ the four-chip ASIC is $32.4\times$ faster than the H200 on the same Pegasus ($d=15$) topology. The per-sweep time for the ASIC implementing the streaming dataflow is flat as K increments, so Eq. (8) returns the same $NII/S = 128$ cycles at every K . The limitation is that the CG will be tall since the IT is a 1D mesh. The FPGA pair illustrates the on-chip memory criterion discussed in Section IV: at $N = 16384$ one FPGA spills to DRAM and sweeps in $130 \mu s$, while two FPGAs hold 8192 spins each in on-chip memory and sweep in $0.68 \mu s$, $191\times$ apart at equal problem size purely due to working-set residency. Fig. 10b compares the platforms in measured energy per attempted spin update. Three orders of magnitude span between the best GPU point and the four-chip ASIC. For scaling up, Eq. (12) makes “low-latency” concrete: a link is fast enough when t_{xfer} fits the latency hiding window. Eq. (13) reads the latency-transparency criterion the other way, as a rule for how large a device must be before a link of a given latency disappears behind it. More bandwidth alone is not sufficient. Using our model, we predict that a Vera Rubin-class GPU still needs $30.7 \mu s$ per sweep at $N=10^6$, only $2.1\times$ faster than the measured H200 despite $4.6\times$ the memory bandwidth.

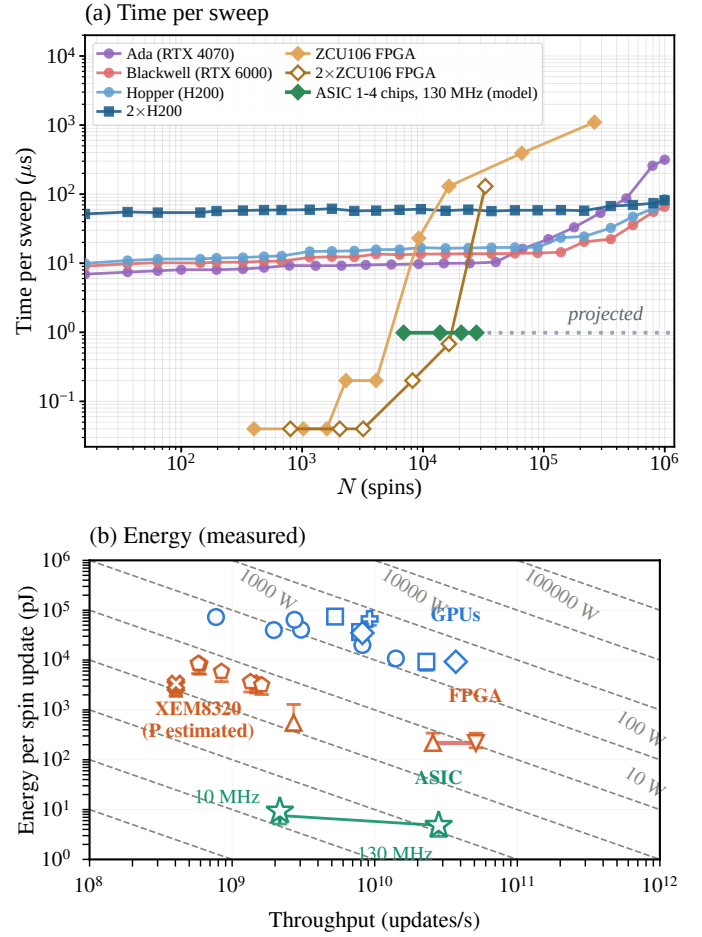


Fig. 10. (a) Per-sweep time against N for the platforms we measure, up to 10^6 spins. Since the ASICs implement a streaming algorithm to hide t_{comm} , adding more chips will keep t_{sweep} constant (extrapolated with dashed line). (b) Measured energy per spin update against sustained throughput; each point carries its accounting range as a vertical bar. Color is the platform class and marker shape the device and its count; the hollow markers and dotted bars are the XEM8320, whose power is a post-route estimate rather than a measurement.

VIII. RELATED WORK

Our decomposition builds on BSP [34], LogP [35], and the roofline model [36], including its variants [55]–[59]. Bounded staleness as a tunable knob comes from Hogwild [51] and SSP [60], with convergence under asynchronous updates established theoretically [30], [61]–[63] and demonstrated on an all-to-all array by Snowball [64]. Aadit et al. [15] measure, per topology, the exchange-to-update ratio above which networked FPGAs match a monolithic reference.

Ising accelerators differ in where couplings sit relative to update logic, from resistive crossbars [65], [66] and near-memory digital arrays [48]–[50], [67], [68] to lattice-coupled and fully connected annealers [13], [14], [32], [69], [70], partitioned simulated bifurcation [71]–[75], and the deeply pipelined Janus family [76]–[80]. Related dataflow accelerators keep the same neighbor-gather-then-local-update structure for graph learning and mathematical programming [81]–[85].

IX. CONCLUSION

We presented the first roofline analysis and a calibrated sweep-time decomposition for distributed digital Ising machines. The model reproduces measured runtime with 4.8% average error across GPUs, FPGAs, and silicon at problem sizes up to one million spins. The roofline analysis identifies low arithmetic intensity, and measurements show that performance degrades sharply when the working set spills beyond on-chip storage. Roofline-guided placement yields up to $4.2\times$ speedup. We proposed two system-level acceleration approaches: bounded-staleness sampling and phase-offset pipelining. Multi-rate bounded staleness and cross-partition bounded staleness reduce TTS by up to $3.16\times$ and $2.17\times$, respectively. Phase-offset pipelining overlaps boundary exchange with interior computation and yields up to $2.01\times$ sweep speedup. A fabricated four-chip 28 nm ASIC sweeps 27,648 spins in $0.985\ \mu\text{s}$ at $4.81\ \text{pJ}$ per update, $32.4\times$ faster than the H200 on the same Pegasus topology.

ACKNOWLEDGEMENTS

This work was supported by the Carnegie Mellon University Department of Electrical and Computer Engineering (CMU ECE). The authors acknowledge the CMU ECE Computing Research GPU Cluster for providing access to NVIDIA H200 systems used in this study.

The streaming ASIC prototype and multi-chip platform were developed with support from CMU ECE research infrastructure and the Apple NSI program. The authors thank Apple for its support.

REFERENCES

- [1] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, vol. 2, p. 5, 2014.
- [2] S. Niazi, N. A. Aadit, M. Mohseni, S. Chowdhury, Y. Qin, and K. Y. Camsari, “Training deep Boltzmann networks with sparse Ising machines,” *Nature Electronics*, vol. 7, pp. 610–619, 2024.
- [3] N. S. Singh, M. Mohseni-Rajaei, S. Niazi, and K. Y. Camsari, “From independent to correlated diffusion: Generalized generative modeling with probabilistic computers,” *arXiv preprint arXiv:2603.27996*, 2026.
- [4] S. Chowdhury, N. A. Aadit, A. Grimaldi, E. Raimondo, A. Raut, P. A. Lott, J. H. Mentink, M. M. Rams, F. Ricci-Tersenghi, M. Chiappini, L. S. Theogarajan, T. Srimani, G. Finocchio, M. Mohseni, and K. Y. Camsari, “Pushing the boundary of quantum advantage in hard combinatorial optimization with probabilistic computers,” *Nature Communications*, vol. 16, p. 9193, 2025.
- [5] E. Ising, “Beitrag zur theorie des ferromagnetismus,” *Zeitschrift für Physik*, vol. 31, no. 1, pp. 253–258, 1925.
- [6] F. Barahona, “On the computational complexity of Ising spin glass models,” *Journal of Physics A: Mathematical and General*, vol. 15, no. 10, pp. 3241–3253, 1982.
- [7] N. Mohseni, P. L. McMahon, and T. Byrnes, “Ising machines as hardware solvers of combinatorial optimization problems,” *Nature Reviews Physics*, vol. 4, no. 6, pp. 363–379, 2022.
- [8] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose, “Quantum annealing with manufactured spins,” *Nature*, vol. 473, no. 7346, pp. 194–198, 2011.
- [9] C. McGeoch and P. Farré, “The D-Wave Advantage system: An overview,” *D-Wave Technical Report*, 2020, d-Wave Systems Inc.
- [10] D-Wave Systems Inc., “Performance gains in the D-Wave Advantage2 system at the 4,400-qubit scale,” *D-Wave White Paper*, 2025, d-Wave Systems Inc.
- [11] T. Honjo, T. Sonobe, K. Inaba, T. Inagaki, T. Ikuta, Y. Yamada, T. Kazama, K. Enbutsu, T. Umeki, R. Kasahara, K. ichi Kwarabayashi, and H. Takesue, “100,000-spin coherent Ising machine,” *Science Advances*, vol. 7, no. 40, p. eabh0952, 2021.
- [12] S. Chowdhury, A. Grimaldi, N. A. Aadit, S. Niazi, M. Mohseni, S. Kanai, H. Ohno, S. Fukami, L. Theogarajan, G. Finocchio, S. Datta, and K. Y. Camsari, “A full-stack view of probabilistic computing with p-bits: devices, architectures, and algorithms,” *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 9, no. 1, pp. 1–11, 2023.
- [13] T. Takemoto, M. Hayashi, C. Yoshimura, and M. Yamaoka, “A $2\times 30\text{k}$ -spin multichip scalable annealing processor based on a processing-in-memory approach for solving large-scale combinatorial optimization problems,” in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019.
- [14] M. Yamaoka, C. Yoshimura, M. Hayashi, T. Okuyama, H. Aoki, and H. Mizuno, “A 20k-spin Ising chip to solve combinatorial optimization problems with CMOS annealing,” *IEEE Journal of Solid-State Circuits*, vol. 51, no. 1, pp. 303–309, 2016.
- [15] N. A. Aadit, X. Zhang, S. Chowdhury, K. Callahan-Coray, K. Lee, S. Bunaiyan, S. Seshan, C. Thomas, J. Twigg, A. Seawright, F. Brewer, T. Srimani, and K. Y. Camsari, “Programmable probabilistic computer with 1,000,000 p-bits,” *arXiv preprint arXiv:2606.25313*, 2026.
- [16] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*. Springer, 1972, pp. 85–103.
- [17] K. Callahan-Coray, K. Lee, K. Jiang, and K. Y. Camsari, “Restoring sparsity in Potts machines via mean-field constraints,” *arXiv preprint arXiv:2602.04200*, 2026.
- [18] D. A. Spielman and N. Srivastava, “Graph sparsification by effective resistances,” in *Proceedings of the fortieth annual ACM symposium on Theory of computing*, 2008, pp. 563–568.
- [19] M. M. H. Sajeeb, N. A. Aadit, S. Chowdhury, T. Wu, C. Smith, D. Chinmay, A. Raut, K. Y. Camsari, C. Delacour, and T. Srimani, “Scalable connectivity for Ising machines: Dense to sparse,” *Physical Review Applied*, vol. 24, no. 1, p. 014005, Jul 2025.
- [20] S. Geman and D. Geman, “Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, no. 6, pp. 721–741, 1984.
- [21] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [22] T. F. Rønnow, Z. Wang, J. Job, S. Boixo, S. V. Isakov, D. Wecker, J. M. Martinis, D. A. Lidar, and M. Troyer, “Defining and detecting quantum speedup,” *Science*, vol. 345, no. 6195, pp. 420–424, 2014.
- [23] S. Boixo, T. F. Rønnow, S. V. Isakov, Z. Wang, D. Wecker, D. A. Lidar, J. M. Martinis, and M. Troyer, “Evidence for quantum annealing with more than one hundred qubits,” *Nature physics*, vol. 10, no. 3, pp. 218–224, 2014.
- [24] T. Albash and D. A. Lidar, “Demonstration of a scaling advantage for a quantum annealer over simulated annealing,” *Physical Review X*, vol. 8, no. 3, p. 031016, 2018.
- [25] M. Kowalsky, T. Albash, I. Hen, and D. A. Lidar, “3-regular three-xorsat planted solutions benchmark of classical and quantum heuristic optimizers,” *Quantum Science and Technology*, vol. 7, no. 2, p. 025008, 2022.
- [26] E. H. L. Aarts and J. Korst, *Simulated Annealing and Boltzmann Machines: A Stochastic Approach to Combinatorial Optimization and Neural Computing*. New York: John Wiley & Sons, 1989.
- [27] K. Selcuk, N. A. Aadit, C. Delacour, J. Q. Silva, N. Sanjay Singh, H. Kaneko, S. Kanai, Y.-J. Wu, Y.-H. Chen, Y.-S. Chen, Y. C. Ong, K.-C. Huang, H. Chuang, H. Ohno, S. Fukami, and K. Y. Camsari, “Dac-free p-bits: Asynchronous self-coloring and on-chip annealing,” in *2025 IEEE International Electron Devices Meeting (IEDM)*, 2025, pp. 1–4.
- [28] N. A. Aadit, A. Grimaldi, M. Carpentieri, L. Theogarajan, J. M. Martinis, G. Finocchio, and K. Y. Camsari, “Massively parallel probabilistic computing with sparse Ising machines,” *Nature Electronics*, vol. 5, no. 7, pp. 460–468, 2022.
- [29] C. Cook, H. Zhao, T. Sato, M. Hiromoto, and S. X.-D. Tan, “Gpu-based Ising computing for solving max-cut combinatorial optimization problems,” *Integration*, vol. 69, pp. 335–344, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167926019301348>

- [30] C. D. Sa, C. Re, and K. Olukotun, "Ensuring rapid mixing and low bias for asynchronous Gibbs sampling," in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 1567–1576.
- [31] J. Gonzalez, Y. Low, A. Gretton, and C. Guestrin, "Parallel Gibbs sampling: From colored fields to thin junction trees," *Artificial Intelligence and Statistics*, pp. 324–332, 2011.
- [32] S. Matsubara, M. Takatsu, T. Miyazawa, T. Shibasaki, Y. Watanabe, K. Takemoto, and H. Tamura, "Digital annealer for high-speed solving of combinatorial optimization problems and its applications," 2020.
- [33] T. Wu, Z. Alsaidan, S. Mukherjee, T.-Y. Lee, D. Xu, A. Raut, H. Tye, A. K. Puthran, K. Selcuk, N. A. Aadit, K. Mai, P. Mohan, K. Y. Camsari, and T. Srimani, "Artifact: Calibration scripts and performance model for distributed digital Ising machines," <https://github.com/WuEric011201/distributed-ising-machines>, 2026.
- [34] L. G. Valiant, "A bridging model for parallel computation," *Communications of the ACM*, vol. 33, no. 8, pp. 103–111, 1990.
- [35] D. Culler, R. Karp, D. Patterson, A. Sahay, K. E. Schauer, E. Santos, R. Subramonian, and T. von Eicken, "Logp: Towards a realistic model of parallel computation," in *PPoPP*, 1993.
- [36] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [37] G. Marsaglia, "Xorshift RNGs," *Journal of Statistical Software*, vol. 8, no. 14, pp. 1–6, 2003.
- [38] N. S. Singh, K. Kobayashi, Q. Cao, K. Selcuk, T. Hu, S. Niazi, N. A. Aadit, S. Kanai, H. Ohno, S. Fukami, and K. Y. Camsari, "Cmos plus stochastic nanomagnets enabling heterogeneous computers for probabilistic inference and learning," *Nature Communications*, vol. 15, no. 1, p. 2685, 2024.
- [39] D. Blackman and S. Vigna, "Scrambled linear pseudorandom number generators," *ACM Transactions on Mathematical Software*, vol. 47, no. 4, pp. 1–32, 2021.
- [40] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, "Parallel random numbers: As easy as 1, 2, 3," in *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, 2011, pp. 1–12.
- [41] B. da Silva, A. Braeken, E. H. D'Hollander, A. Touhafi, J. G. Cornelis, and J. Lemeire, "Performance modeling for FPGAs: Extending the roofline model with high-level synthesis tools," in *International Journal of Reconfigurable Computing*, vol. 2013, 2013, p. 428078.
- [42] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
- [43] V. Volkov, "Understanding latency hiding on GPUs," Ph.D. dissertation, University of California, Berkeley, 2016, tech. Rep. UCB/EECS-2016-143.
- [44] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proc. Conf. High Performance Computing Networking, Storage and Analysis (SC)*, 2009, pp. 1–11.
- [45] S. Filippone, V. Cardellini, D. Barbieri, and A. Fanfarillo, "Sparse matrix-vector multiplication on GPGPUs," *ACM Transactions on Mathematical Software*, vol. 43, no. 4, 2017.
- [46] J. R. Rice and R. F. Boisvert, *Solving Elliptic Problems Using ELLPACK*, ser. Springer Series in Computational Mathematics. New York: Springer-Verlag, 1985, vol. 2.
- [47] K. Boothby, P. Bunyk, J. Raymond, and A. Roy, "Next-generation topology of D-Wave quantum processors," 2020, arXiv preprint arXiv:2003.00133.
- [48] S. R. Sundara Raman, L. K. John, and J. P. Kulkarni, "SACHI: A stationarity-aware, all-digital, near-memory, Ising architecture," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 719–731.
- [49] H.-W. Chiang, C.-F. Nien, H.-Y. Cheng, and K.-P. Huang, "ReAIM: A ReRAM-based adaptive Ising machine for solving combinatorial optimization problems," in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024, pp. 58–72.
- [50] G. Yang, S. Karimi, C. A. Ríos Ocampo, A. K. Coskun, and A. Joshi, "SOPHIE: A scalable recurrent Ising machine using optically addressed phase change memory," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1548–1561.
- [51] F. Niu, B. Recht, C. Ré, and S. J. Wright, "Hogwild!: A lock-free approach to parallelizing stochastic gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 693–701.
- [52] D. Perera *et al.*, "Chook: A comprehensive suite for generating binary optimization problems with planted solutions," *arXiv preprint arXiv:2005.14344*, 2020.
- [53] D. Perera *et al.*, "Computational hardness of spin-glass problems with tile-planted solutions," *Physical Review E*, vol. 101, no. 2, p. 023316, 2020.
- [54] D. Iyer and S. Achour, "Efficient optimization with encoded Ising models," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 85–98.
- [55] A. Ilic, F. Pratas, and L. Sousa, "Cache-aware roofline model: Upgrading the loft," vol. 13, no. 1, 2014, pp. 21–24.
- [56] A. Lopes, F. Pratas, L. Sousa, and A. Ilic, "Exploring GPU performance, power and energy-efficiency bounds with cache-aware roofline modeling," in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2017, pp. 259–268.
- [57] C. Yang, T. Kurth, and S. Williams, "Hierarchical roofline analysis for GPUs: Accelerating performance optimization for the NERSC-9 Perlmutter system," vol. 32, no. 20, 2020, p. e5547.
- [58] C. Yang, Y. Wang, S. Farrell, T. Kurth, and S. Williams, "Hierarchical roofline performance analysis for deep learning applications," in *Proceedings of the Computing Conference (Intelligent Computing, IC 2021)*, ser. Lecture Notes in Networks and Systems. Springer, 2021.
- [59] N. Ding and S. Williams, "An instruction roofline model for GPUs," in *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2019, pp. 7–18.
- [60] Q. Ho, J. Cipar, H. Cui, S. Lee, J. K. Kim, P. B. Gibbons, G. A. Gibson, G. R. Ganger, and E. P. Xing, "More effective distributed ML via a stale synchronous parallel parameter server," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013.
- [61] A. Terenin, D. Simpson, and D. Draper, "Asynchronous Gibbs sampling," in *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 108, 2020, pp. 144–154.
- [62] M. X. Burns and M. C. Huang, "Limitations in parallel Ising machine networks: Theory and practice," *Physical Review Applied*, vol. 24, no. 2, p. 024045, 2025, arXiv:2503.17548.
- [63] N. Onizawa and T. Hanyu, "A unified performance-cost landscape of parallel p-bit Ising machines based on update dynamics," *arXiv preprint arXiv:2604.01564*, 2026.
- [64] S. Hong, K. Jeong, and T. Jang, "Snowball: A scalable all-to-all Ising machine with dual-mode Markov chain Monte Carlo spin selection and asynchronous spin updates," *arXiv preprint arXiv:2601.21058*, 2026.
- [65] R. Afoakwa, Y. Zhang, U. K. R. Vengalam, Z. Ignjatovic, and M. Huang, "Brim: Bistable resistively-coupled Ising machine," pp. 749–760, 2021.
- [66] A. Sharma, R. Afoakwa, Z. Ignjatovic, and M. Huang, "Increasing Ising machine capacity with multi-chip architectures," in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2022, pp. 508–521.
- [67] Y. Wu, J. Bae, S. Shin, and B. Kim, "COBI: A degree-of-56 column-bipartite densely connected digital Ising chip with 8b spin coefficients," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2026, p. 10.8.
- [68] Y. Fu, A. Fan, W. Yue, H. Zhao, D. Shi, Q. Wu, J. Li, X. Zhang, Y. Tao, Y. Yang, and B. Yan, "PROCA: Programmable probabilistic processing unit architecture with accept/reject prediction and multicore pipelining for causal inference," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 761–774.
- [69] M. Aramon, G. Rosenberg, E. Valiante, T. Miyazawa, H. Tamura, and H. G. Katzgraber, "Physics-inspired optimization for quadratic unconstrained problems using a digital annealer," *Frontiers in Physics*, vol. 7, p. 48, 2019.
- [70] K. Yamamoto, K. Kawamura, K. Ando, N. Mertig, T. Takemoto, M. Yamaoka, H. Teramoto, A. Sakai, S. Takamaeda-Yamazaki, and M. Motomura, "STATICA: A 512-spin 0.25M-weight annealing processor with an all-spin-updates-at-once architecture for combinatorial optimization with complete spin-spin interactions," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 1, pp. 165–178, 2021.
- [71] H. Goto, K. Tatsumura, and A. R. Dixon, "Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems," *Science Advances*, vol. 5, no. 4, p. eaav2372, 2019.

- [72] H. Goto, K. Endo, M. Suzuki, Y. Sakai, T. Kanao, Y. Hamakawa, R. Hidaka, M. Yamasaki, and K. Tatsumura, “High-performance combinatorial optimization based on classical mechanics,” *Science Advances*, vol. 7, no. 6, p. eabe7953, 2021.
- [73] K. Tatsumura, A. R. Dixon, and H. Goto, “FPGA-based simulated bifurcation machine,” in *International Conference on Field Programmable Logic and Applications (FPL)*, 2019, pp. 59–66.
- [74] K. Tatsumura, M. Yamasaki, and H. Goto, “Scaling out Ising machines using a multi-chip architecture for simulated bifurcation,” *Nature Electronics*, vol. 4, no. 3, pp. 208–217, 2021.
- [75] T. Kashimata, M. Yamasaki, R. Hidaka, and K. Tatsumura, “Efficient and scalable architecture for multiple-chip implementation of simulated bifurcation machines,” *arXiv preprint arXiv:2311.17370*, 2023.
- [76] M. Baity-Jesi *et al.*, “Reconfigurable computing for Monte Carlo simulations: Results and prospects of the Janus project,” *The European Physical Journal Special Topics*, vol. 210, pp. 33–51, 2012.
- [77] M. Baity-Jesi, R. A. Baños, A. Cruz, L. A. Fernandez, J. M. Gil-Narvion, A. Gordillo-Guerrero, D. Iñiguez, A. Maiorano, F. Mantovani, E. Marinari, V. Martin-Mayor, J. Monforte-Garcia, A. M. Sudupe, D. Navarro, G. Parisi, S. Perez-Gavio, M. Pivanti, F. Ricci-Tersenghi, J. J. Ruiz-Lorenzo, S. F. Schifano, B. Seoane, A. Tarancón, R. Tripiccion, and D. Yllanes, “Janus II: A new generation application-driven computer for spin-system simulations,” *Computer Physics Communications*, vol. 185, no. 2, pp. 550–559, 2014.
- [78] C. Yoshimura, M. Hayashi, T. Okuyama, and M. Yamaoka, “Implementation and evaluation of FPGA-based annealing processor for Ising model by use of resource sharing,” *International Journal of Networking and Computing*, vol. 7, no. 2, pp. 154–172, 2017.
- [79] S. Patel, L. Chen, P. Canoza, and S. Salahuddin, “Ising model optimization problems on a FPGA accelerated restricted Boltzmann machine,” *arXiv preprint arXiv:2008.04436*, 2020.
- [80] H. Gyoten, M. Hiromoto, and T. Sato, “Area efficient annealing processor for Ising model without random number generator,” *IEICE Transactions on Information and Systems*, vol. E101.D, no. 2, pp. 314–323, 2018.
- [81] R. Song, C. Wu, C. Liu, A. Li, M. Huang, and T. Geng, “DS-GL: Advancing graph learning via harnessing nature’s power within scalable dynamical systems,” in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024, pp. 45–57.
- [82] C. Wu, R. Song, C. Liu, Y. Yang, A. Li, M. Huang, and T. Geng, “DS-TPU: Dynamical system for on-device lifelong graph learning with nonlinear node interaction,” in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2025, pp. 1867–1879.
- [83] C. Liu, G. Sun, C. Wu, R. Song, A. Li, M. Huang, and T. Geng, “DS-TIDE: Harnessing dynamical systems for efficient time-independent differential equation solving,” in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2025, pp. 1690–1703.
- [84] S. R. Sundara Raman, L. K. John, and J. P. Kulkarni, “SPARK: Sparsity aware, low area, energy-efficient, near-memory architecture for accelerating linear programming problems,” in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 99–112.
- [85] M. Wang, I. McInerney, B. Stellato, and F. Tu, “Multi-issue butterfly architecture for sparse convex quadratic programming,” in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1574–1587.